

Breaking DES with side-channel attacks

TSIT03 Cryptology

Jonathan Jogenfors
jonathan.jogenfors@liu.se

Niklas Johansson
niklas.johansson@liu.se

September 16, 2019

This lab will demonstrate how power analysis of cryptographic hardware can reveal the key. We will be using basic electronic measurement tools such as oscilloscopes to demonstrate this side-channel attack.

Contents

1	Basic scenario and goal of the study	2
2	To be done before the lab	2
3	To be done when starting the lab	2
3.1	Setting up the equipment	2
3.2	Configuring the oscilloscope	5
4	Hacking the DES key	6
4.1	Finding the first bit value	6
4.2	Finding the rest of the key	7
4.3	Decrypting with the found key	8
5	Finishing the lab	8
A	The Data Encryption Standard	10
B	The hardware board	10
B.1	Command reference	10
C	Bit value reference	12
D	Source code	14
D.1	Pseudocode implementation	14
D.2	C implementation	16

1 Basic scenario and goal of the study

You will be using a small hardware board (fig. 1) with a generic microprocessor programmed to perform DES encryption and decryption. The scenario is that you are the attacker and want to find out the secret key stored inside the board. There is no way of getting to the key directly, so you will need to perform a side-channel attack by measuring the power consumption of the board while the algorithm is running. The hardware board also allows the user to load a custom key in order to compare the power consumption.

2 To be done before the lab

Read the description of the DES algorithm including the implementation details. Familiarize yourself with the appendix chapters and try to understand the oscilloscope, hardware board, computer commands as well as you can.

Print a copy of this lab PM so that you have a physical copy with you. There is an important hint given regarding the bits of the secret key. Find this hint in the lab PM. This hint makes it easier to find the secret key and immediately gives you the values of the first, second, third and last column of table 2 by looking at table 3. Fill in these columns on your printed copy of the lab PM. Also, bring your LiU card for identification purposes.

3 To be done when starting the lab

Go to the course homepage and check to see if there is a newer version of the lab PM available.

3.1 Setting up the equipment

The workbench is equipped with an oscilloscope and a computer. You will be provided with one of the following items:

1. One hardware board (fig. 1)
2. One power supply (black)
3. One probe (for the TRIG port)
4. One black probe (for the AVCC port)
5. One serial-to-usb adapter

Set up the hardware in the following way:

1. Connect one of the probes to CH1 on the oscilloscope
2. Connect the ground clip of the probe to the GND pin on the board
3. Connect the measuring clip of the probe to the TRIG pin on the board
4. Connect the second probe to CH2 on the oscilloscope

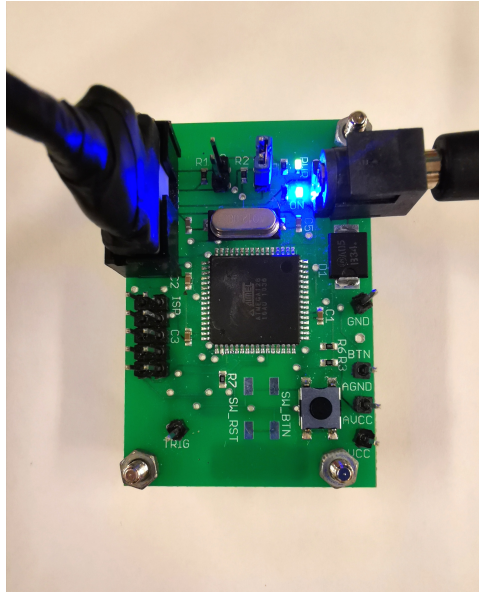


Figure 1: The hardware board

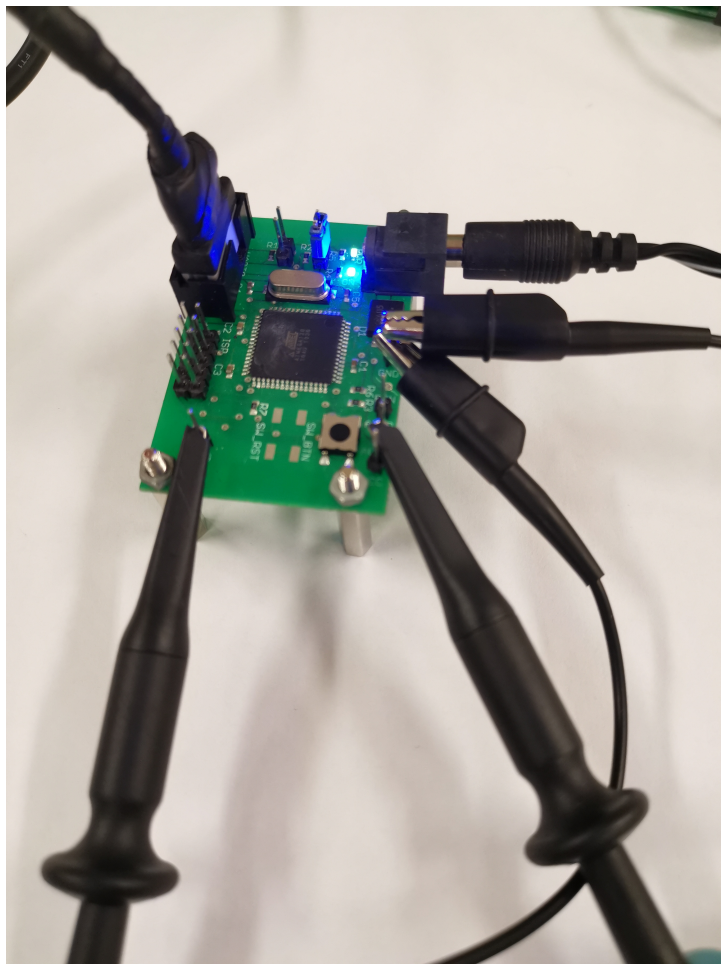


Figure 2: How to connect the probes. Note that the color of the probe might be different from what you have.

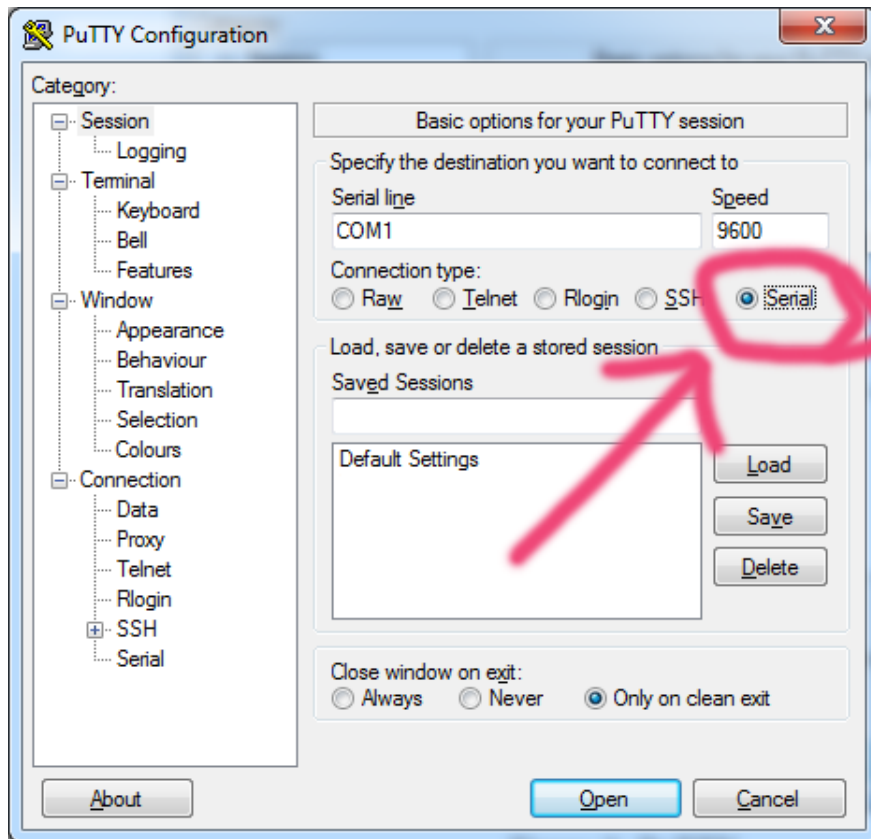


Figure 3: Configuring PuTTY. Do not be concerned with your window looking slightly different.

5. Connect the ground clip of the second probe to the GND pin on the board
6. Connect the measuring clip of the second probe to the AVCC pin on the board. This pin is the middle one of the group of three.
7. Connect the USB-to-serial cable to the board and to the USB outlet on back of the screen.
8. Do not connect the power adapter yet!

The board should look like fig. 2. Do not connect the power plug to the outlet yet! If you already have powered up the board, disconnect it again.

On the computer, start the program called PuTTY. In the configuration dialog, set the connection type to **Serial** as shown in fig. 3. You need to change the serial line from COM1 to the actual number. Try finding this number in the "Device Manager", or try COM3 and click on **Open**. If this gives an error message, restart PuTTY and try again with COM4, or COM5 and so on. When it works you should get a black window. When you see a black window, connect the power adapter to the board. You should immediately see the following message appear in the PuTTY window:

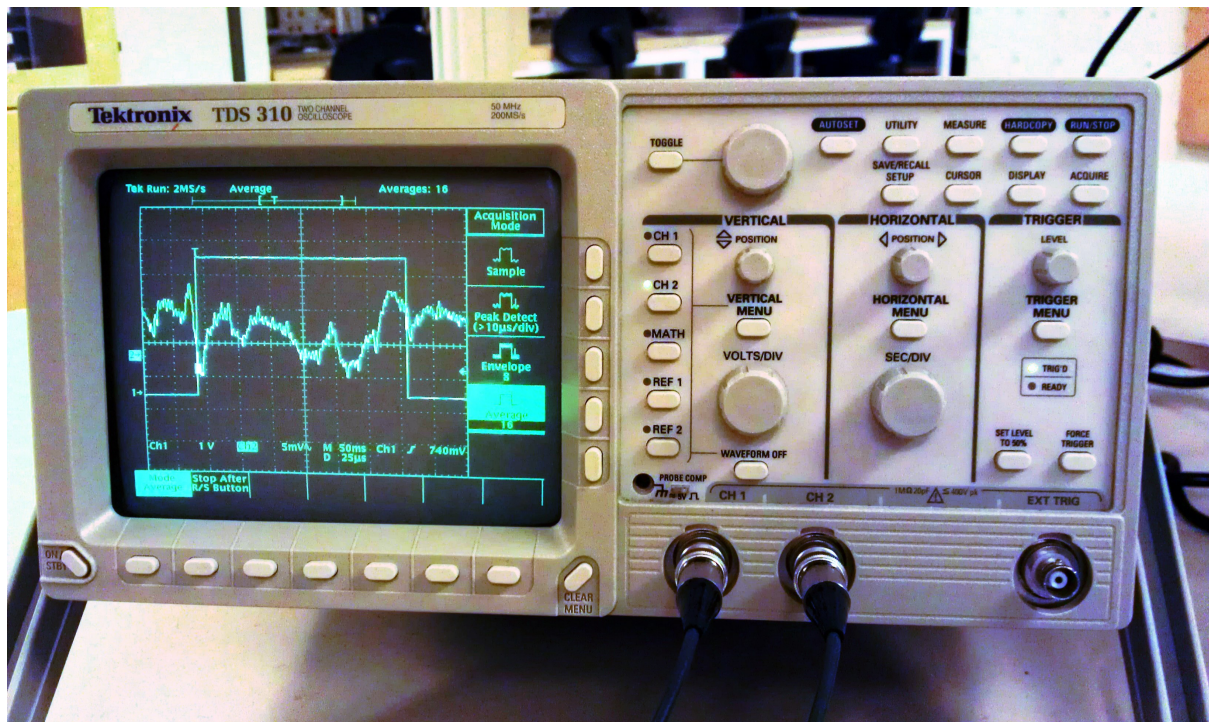


Figure 4: Tektronix TDS 310 Oscilloscope

Kryptolab v1.0

```
=====
Board ID: <number>
1. DES round recognition
2. DES key recovery
3. Power analysis of load/mult instruction sequence
=====
```

Choose a task:

Choose DES key recovery. Read through the command reference in appendix B.1 and familiarize yourself with the commands. Click r to make it start running the DES algorithm over and over.

3.2 Configuring the oscilloscope

Now you need to set up the oscilloscope (fig. 4). The On/Off button is below the screen to the left. It takes a little while for the oscilloscope to start up, but when it turns on it should say “Passed” on the screen. If it does not, or if you see “+++” anywhere on the screen it means that it has detected a malfunction. If this is the case, contact the lab assistant.

If the power-on self test passed, continue with the configuration.

- Click on the SAVE/RECALL SETUP button and perform a factory reset by clicking on the soft button labeled OK Confirm Factory Init just to the right of the screen.

This ensures that any previous configurations are reset.

- Now, click on **CH1** followed by **TRIGGER MENU**. Click on the button below the screen that says **Level** followed by the button on the right side of the screen that says **Set to TTL**. Also, set the **Mode** (below the screen) to **Normal** (to the right of the screen).
- Next we set up the second channel. Click on **CH2** and then **VERTICAL MENU**. To the right of the screen, click **AC $\sim$** , which selects the AC coupling mode.
- Now click the **ACQUIRE** button in the top right corner and select **Average** on the button to the right of the screen. When this is selected you can use the blank rotating dial located on the upper side to the right of the screen. When you operate the dial, note that the number right under **Average** changes between different powers of 2 (8, 16, 32 etc.). For now, choose the value 32. This setting makes the oscilloscope display the average of the last 32 cycles instead of just the last in order to reduce noise. If you want faster response, choose a lower value.

It is sometimes hard to find the interesting part of the spectrum. Good starting parameters may be to put CH 1 to 2 V/DIV, CH 2 to 5 mV/Div, and horizontal resolution to 50 μ s SEC/DIV.

There are two ways to scroll horizontally. First, enable Delay mode: Click on **HORIZONTAL MENU** and then click the **CH 1** button. On the right side of the screen, select **Delayed Only**. Now you can control the delay by operating the blank knob. Try it! This is the coarse scrolling. For fine-tuning, use the **POSITION** knob.

Now you have everything you need to start the actual lab.

4 Hacking the DES key

Turn your attention to the oscilloscope. The board sets the **TRIG** signal to high when running the algorithm and clears it when finished with this step. Click on **CH1** to select the first channel (the corresponding LED will light up) and then turn the knobs **Volts/DIV** and **SEC/DIV** to change the zoom level. On the bottom of the screen you will see numbers that tell you what voltage/time corresponds to one box. Above the knobs you will also find corresponding smaller knobs labeled **position** that will move the signal along the x and y axes. Try to zoom in so that one **TRIG** pulse almost fills the entire window.

4.1 Finding the first bit value

Zoom in on the **TRIG** pulse so that it fills the window. If you see no **TRIG** pulse, be sure that the chip is running the key (press **r**). Channel 2 of the oscilloscope is connected to the **AVCC** port, which will give us information about the power consumption. Click on the **CH2** button on the oscilloscope. Now we can modify the zoom level of this channel. Change the **VOLTS/DIV** until you see a comb-like pattern on channel 2 like in fig. 4. Note that changing the **SEC/DIV** always affect both channels!

Now try changing the first bit of the DES key. To change the value of a bit, use the commands **c** and **s** or **C** or **S**. The DES algorithm does, however not use the bits in an

ordered fashion, so the bit that is seen first on the oscilloscope is bit no. 56 (see the table below). To set the value of this bit to 1, type

```
s 56
```

and to set the value to 0, type

```
c 56
```

back and forth. You should see a tiny bit of the waveform move up and down slowly as you do this. Can you see it? If it is difficult to see, try zooming in on the first part of the TRIG pulse.

Tip: You don't have to type "56" after each set and clear command. Instead, use the capitalized versions **C** and **S**; they will remember which bit you are working on. You can even hold down the **SHIFT** key on the keyboard and press **C** and **S** back and forth. Don't press too quickly, though, as we have enabled the averaging feature of the oscilloscope which means it takes some time for the image to change.

Show to the lab assistant

The changing of the bit value can be seen on the oscilloscope as a lowering and increasing of amplitude in a specific part of the cycle. Do you see it? If not, talk to the lab assistant.

Now that you see the first bit moving up and down, start to compare this to the secret key. Switch back and forth between the secret key (type **x**) and the user key (command **r**). Look at the first bit and see if it changes between the keys. If it does, change bit value no. 56 and try again. You want to match the user key and the secret key at all positions.

4.2 Finding the rest of the key

Load in the hint from the preparatory exercise with the command (l stands for "load")

```
l 6060606060606060
```

This is done to make sure all the previous bits (including no. 56) are set to the correct values. Now follow the order in table 1 and try to find the first unknown bit value on the oscilloscope screen. You will need to scroll quite far, so the **POSITION** knob will not be able to scroll far enough. Instead, click on **HORIZONTAL MENU** and turn the blank knob. While you turn this knob, the screen will be noisy, but this goes away when you stop the turn. Turn the knob while toggle the value of bit 59 until you see the bit moving on the screen (make sure that you are running in **user key** mode).

The flow for finding a bit is now:

1. Change to the user key with **r**.
2. Make sure the previous bit is set correctly
3. Check table 1 for the next bit.

4. Flip this bit back and forth.
5. Click on **HORIZONTAL MENU** to control the delay with the blank knob.
6. Find the position of the bit on the oscilloscope (the one that moves when you change its value).
7. Make sure you see the moving bit clearly (zoom a little if needed)
8. Switch to the secret key with **x**
9. Find the bit value of the user key.
10. Set the user key bit to the correct value you just found.
11. Write down the bit value on paper.
12. Go to step 1.

Teamwork is essential! It is recommended to share the workload, one person controls the oscilloscope, one person reads table 1, one person controls the computer and one person writes the bit values into table 2. Don't forget to write down on paper so that you don't lose your progress if you accidentally erase the key.

Do not continue if you are unsure of a bit value as it should be quite clear on the oscilloscope. Ask the lab assistant for help if you need.

Now for a hint: The key consists of the numbers 0 to 9 encoded with the US-ASCII encoding, see table 3. With this knowledge you should be able to fill in the first, second, third and last column of table 2. It is important that you fill in these bits correctly, otherwise you won't have a chance of completing the lab on time!

4.3 Decrypting with the found key

When you have found all the bits you can move to the final step. Press **d** to decrypt the ciphertext and retrieve the cleartext. You will see a hex representation of an English word in US-ASCII. Translate the cleartext into English with table 4. If the values aren't found in the table it means that the key is incorrect. Probably you just made a small mistake somewhere, which is easy to fix. The key is made to be simple to find and to have a pattern. Perhaps you can find the incorrect bit in this way, otherwise ask the assistant.

5 Finishing the lab

In order to finish the lab you need to show the lab assistant the plaintext and how you cracked the system. No report is required. Good luck!

Step 1: 56 (known)	Step 20: 34 (known)	Step 39: 45
Step 2: 48 (known)	Step 21: 26 (known)	Step 40: 37
Step 3: 40 (known)	Step 22: 18 (known)	Step 41: 29
Step 4: 32 (known)	Step 23: 10 (known)	Step 42: 21
Step 5: 24 (known)	Step 24: 02 (known)	Step 43: 13
Step 6: 16 (known)	Step 25: 59	Step 44: 05
Step 7: 08 (known)	Step 26: 51	Step 45: 60
Step 8: 00 (known)	Step 27: 43	Step 46: 52
Step 9: 57 (known)	Step 28: 35	Step 47: 44
Step 10: 49 (known)	Step 29: 62	Step 48: 36
Step 11: 41 (known)	Step 30: 54	Step 49: 28
Step 12: 33 (known)	Step 31: 46	Step 50: 20
Step 13: 25 (known)	Step 32: 38	Step 51: 12
Step 14: 17 (known)	Step 33: 30	Step 52: 04
Step 15: 09 (known)	Step 34: 22	Step 53: 27
Step 16: 01 (known)	Step 35: 14	Step 54: 19
Step 17: 58 (known)	Step 36: 06	Step 55: 11
Step 18: 50 (known)	Step 37: 61	Step 56: 03
Step 19: 42 (known)	Step 38: 53	

Table 1: DES does not use the key bit in order. The first bit you see on the oscilloscope is no. 56, followed by no. 48 and then 40 and so on. The full order of all bits is presented here.

Bit number	7	6	5	4	3	2	1	0
0 to 7								
8 to 15								
16 to 23								
24 to 31								
32 to 39								
40 to 47								
48 to 55								
56 to 63								

Table 2: Helpful table for finding out the DES1 key. Be sure to fill in the known bit values before continuing! Each row represents one ASCII-coded number in the key, i.e., the key is built from eight concatenated bytes. Each ASCII-coded byte has its LSB to the right, but the bits of the key is enumerated from left to right and up to down.

A The Data Encryption Standard

DES (short for *Data Encryption Standard*) is a symmetric-key encryption algorithm. It uses a 56-bit key and was standardized in the 70's. Today, DES is obsolete and considered insecure. A modern and much more secure replacement is AES. A general schematic of DES is found in fig. 5. The F-boxes are depicted in fig. 6.

B The hardware board

Three versions of DES are implemented on the board, and each version uses a different secret key for you to find out. The most straightforward is DES2, which implements a naïve version of the DES algorithm with no protection against power analysis. For the most part you will be using DES1 which is the same as DES2 except that it on purpose exaggerates the power analysis by performing eight NOP instructions for every bit set to 1. A NOP instuction simply make the processor do nothing for one cycle, allowing us to distinguish between the different states of the algorithm. Finally we have DES3 which is hardened against power analysis by making sure the different code paths take exactly the same number of clock cycles. Cracking DES3 is very hard and probably impossible due to the noise levels.

B.1 Command reference

When you have started the board you will be in the main menu as described above. You will only need to use the `DES round recognition` and `DES key recovery` commands for this lab.

If you press 1 you will get to the following menu:

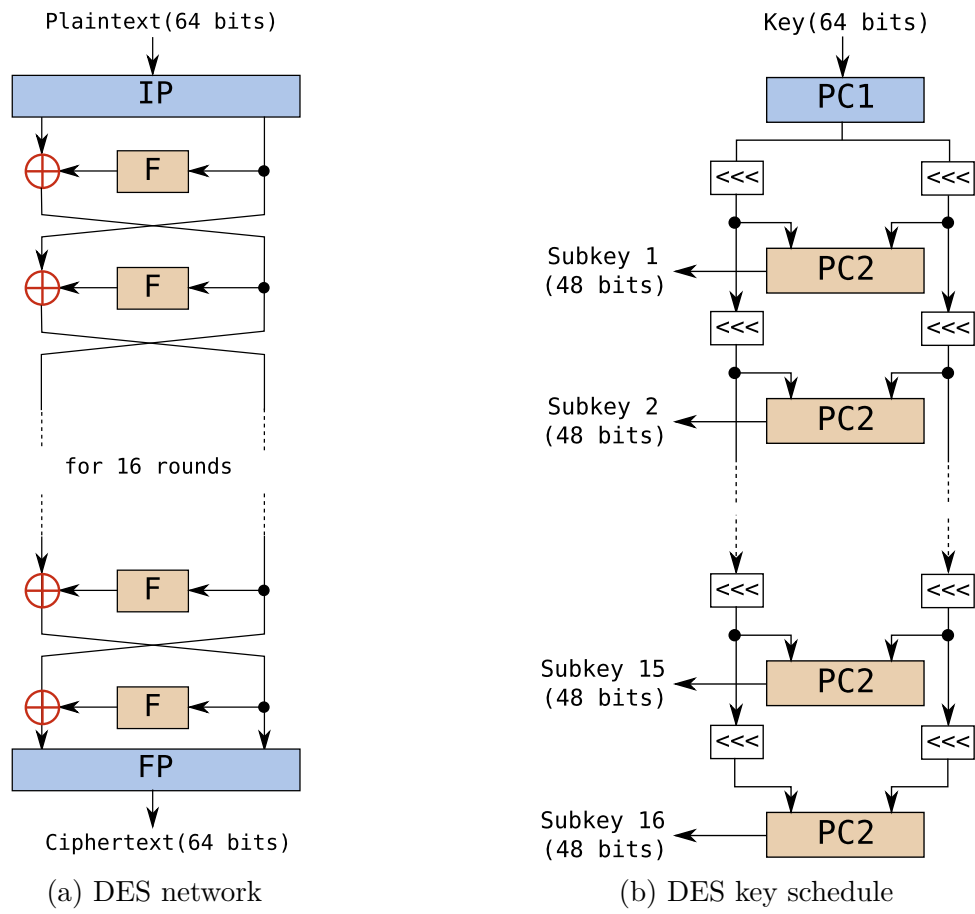


Figure 5: Schematic view of the DES algorithm

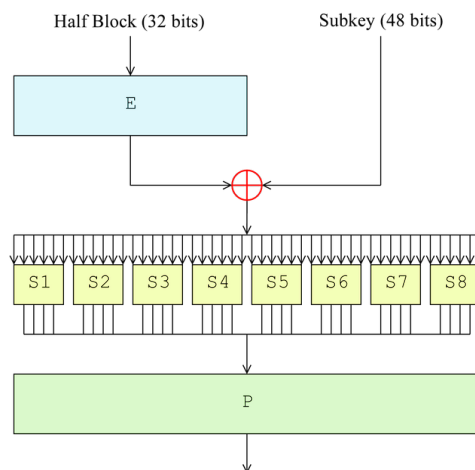


Figure 6: One F-box

```
=====
1. Run once
2. Run repeatedly until any button is pressed
9. Quit
=====
```

In this step the board will run the rounds over and over again in order for you to identify the sixteen DES rounds.

The main program is called **DES key recovery** and is started if you press 2. You will be greeted by a short help text (note that the help text is slightly modified from the actual program in order to make them more understandable).

```
=====
h                print this help
l 0123456789abcdef set the user key to 0123456789abcdef (hex only!)
z                set the user key to 0000000000000000 (all zeros)
o                set the user key to ffffffffffffffff (all ones)
s n              set the n:th bit of the user key to 1
c n              set the n:th bit of the user key to 0
S (Capital S!)  set the bit from the previous command to 1
C (Capital C!)  set the bit from the previous command to 0
p                print the user key
x                run DES with the secret key
r                run DES with the user key
d                decrypt the ciphertext with the user key
q                quit to main menu
1                use DES1 (easy)
2                use DES2 (hard)
3                use DES3 (expert mode!)
=====
```

As soon as **DES key recovery** is started, the board will run the algorithm over and over again. You can make it use either the secret key or the user key with the commands **x** and **r**. You can modify the user key in several ways. The commands **z** and **o** sets all bits in the user key to 0 and 1, respectively. You can change individual bits with the commands **c** and **s**. If you want to switch between two bit values without having to enter the indices every time, the commands **C** and **S** will be useful.

C Bit value reference

DES discards the rightmost bit in every key byte. Therefore, you need to use the values in table 3 when you enter your key into the program. The standard US-ASCII encoding of all latin letters are found in table 4.

Decimal	Binary representation	Left-shifted to 8 bits	Hexadecimal
0	0110000	01100000	60
1	0110001	01100010	62
2	0110010	01100100	64
3	0110011	01100110	66
4	0110100	01101000	68
5	0110101	01101010	6A
6	0110110	01101100	6C
7	0110111	01101110	6E
8	0111000	01110000	70
9	0111001	01110010	72

Table 3: US-ASCII-encoded numbers 0 to 9 as used in the DES key

Letter	Hexadecimal	Letter	Hexadecimal
A	41	a	61
B	42	b	62
C	43	c	63
D	44	d	64
E	45	e	65
F	46	f	66
G	47	g	67
H	48	h	68
I	49	i	69
J	4A	j	6A
K	4B	k	6B
L	4C	l	6C
M	4D	m	6D
N	4E	n	6E
O	4F	o	6F
P	50	p	70
Q	51	q	71
R	52	r	72
S	53	s	73
T	54	t	74
U	55	u	75
V	56	v	76
W	57	w	77
X	58	x	78
Y	59	y	79
Z	5A	z	7A
space	20		

Table 4: List of Latin letters (including space) with hexadecimal US-ASCII codes.

D Source code

In this section we list the source code for the hardware board.

D.1 Pseudocode implementation

The permutations and S-boxes are implemented as tables. As seen in fig. 5, we call these IP, PC1, PC2, E, P, FP and S_x, where x is the S-box. ROTTABLE indicates if k should be rotated one or two steps in each round before creating the subkey. The following variables are used:

in the 64-bit plaintext input

out the 64-bit ciphertext output

key is the 64-bit original key

k is the permuted 56-bit key used in the creation of round keys

kr is the current round key

L is the left half-block used during encryption

R is the right half-block used during encryption

Halfblock is the half-block currently treated in the F function

DATA is a table for the 8 variables used in the S-boxes

In the pseudocode we have used “:=” to indicate assignment and “=” means comparison.

Note that the implementation performs eight iterations involving two rounds each instead of one round of 16 iterations.

Source code pseudocode.txt

```
1 R:= pickout_rightmost_32 bits (in);
2 L:= pickout_leftmost_32 bits (in);
3 Set_trig_signal;
4 k:=0;
5 i:=0;
6 j:=1
7 while i<56
8 {
9   i:=i+1;
10  // Comment: Here we start a bitwise treatment
11  // of the key
12  if key[PC1[i]] = 1
13  {
14    k:= k XOR j;
15    if DES1
16    {
```

```

17      // Comment: If we are using
18      // DES1 then perform eight
19      // idle cycles to exaggerate
20      // the difference between 0 and 1
21      NOP; NOP; NOP; NOP;
22      NOP; NOP; NOP; NOP;
23  }
24  }
25  else
26  {
27      if DES3
28      {
29          // Comment: If we are using DES3 we
30          // perform one extra idle loop here
31          // in order to harden the system
32          // against side-channel attacks
33          NOP;
34      }
35  }
36  j:=leftshift(j);
37  }
38  Clear_trig_signal;
39  //Comment: End permutation phase, start encryption phase
40  i:=0;
41  while i<8
42  {
43      i:=i+1;
44      k:=leftrotate(k);
45      if ROTTABLE[2*i-1] = 2 then k:= leftrotate(k);
46      kr:=permute(PC2, k);
47      Halfblock:=permute(E, R);
48      Halfblock:=Halfblock XOR kr;
49      j:=0;
50      while j<7
51      {
52          j:=j+1
53          DATA[j]:=Sixbitpart(Halfblock, j);
54          DATA[j]:=Sj[DATA[j]];
55      }
56      Halfblock:=permute(P, DATA);
57      L:=L XOR Halfblock;
58      /Comment: one round finished;
59      k:=leftrotate(k);
60      if ROTTABLE[2*i] = 2 then k:= k leftrotate(k);
61      kr:=permute(PC2, k);
62      Halfblock:=permute(E, L);
63      Halfblock:=Halfblock XOR kr;

```

```

64     j:=0;
65     while j<7
66     {
67         j:=j+1
68         DATA[j]:=Sixbitpart(r, j);
69         DATA[j]:=Sj[DATA[j]];
70     }
71     Halfblock:=permute(P, DATA);
72     R:=R XOR Halfblock;
73 }
74 out:=permute(FP, L, R);

```

D.2 C implementation

If you are interested you can also study the actual C code as shown below.

Source code des.c

```

1  /*
2  This file is part of the AVR-Crypto-Lib.
3  Copyright (C) 2008 Daniel Otte (daniel.otte@rub.de)
4  (Slight modifications done by Simon Johansson and Andreas
5  Ehliar for the security course at LiU)
6
7  This program is free software: you can redistribute it and/or
8  modify it under the terms of the GNU General Public License as
9  published by the Free Software Foundation, either version 3 of
10 the License, or (at your option) any later version.
11
12 This program is distributed in the hope that it will be useful,
13 but WITHOUT ANY WARRANTY; without even the implied warranty of
14 MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See
15 the GNU General Public License for more details.
16
17 You should have received a copy of the GNU General Public License
18 along with this program. If not, see <http://www.gnu.org/licenses/>.
19 */
20
21 // Compilation options:
22 // * DES1: Compiled without optimizations (with extra NOPs added below)
23 //       Leads to a huge difference in time between a one and zero
24 // * DES2: Compiled without optimizations (with no extra NOPs)
25 //       Leads to a small difference in time between a one and zero
26 // * DES3: Compiled with optimization.
27 //       By chance this leads to constant execution time since a
28 //       conditional branch in the selected processor takes two clock
29 //       cycles whereas an untaken branch and an or instruction also

```

```

30 //          takes two clock cycles.
31 const uint8_t bit_table[8] = {0x80, 0x40, 0x20, 0x10, 0x08, 0x04,
    0x02, 0x01};
32 void permute(prog_uint8_t *ptable, const uint8_t *in, uint8_t *out){
33     uint8_t ib, ob; /* in-bytes and out-bytes */
34     uint8_t byte, bit; /* counter for bit and byte */
35     ib = pgm_read_byte(&(ptable[0]));
36     ob = pgm_read_byte(&(ptable[1]));
37     ptable = &(ptable[2]);
38     uint8_t pos = 0;
39     for(byte=0; byte<ob; ++byte){
40         uint8_t x,t=0;
41         for(bit=0; bit<8; ++bit){
42             x=pgm_read_byte(&(ptable[byte*8+bit])) -1 ;
43             t<<=1;
44
45             //Bit position independent version which (at
46             //least with current AVR compiler gives
47             //constant execution time unless you compile
48             //without optimization)
49             if((in[x/8] & bit_table[x%8]) {
50                 t|=0x01; // ori is one instruction
51 #ifdef DES1
52                 asm volatile("nop");
53                 asm volatile("nop");
54                 asm volatile("nop");
55                 asm volatile("nop");
56                 asm volatile("nop");
57                 asm volatile("nop");
58                 asm volatile("nop");
59                 asm volatile("nop");
60                 asm volatile("nop");
61                 asm volatile("nop");
62 #endif
63             }
64             pos++;
65         }
66         out[byte]=t;
67     }
68 }

```